

LEETCODE PROBLEMSET 53. MAXIMUM SUBARRAY SOLUTION

I. USING BRUTE FORCE - ORIGINAL SOLUTION:

1. Modeling:

• Start:

$$i = 0$$

$$j = 0 \dots n-1$$

$$k = 0 \dots j$$

• • •

$$i = n-2$$

$$j = n-2 \dots n-1$$

$$k = n-2 \dots n-1$$

• End:

$$i = n-1$$

$$j = n-1$$

$$k = n-1$$

```
int maxSum = -∞;
for (int i=0; i<n; i++) {
    for (int j=i; j<n; j++) {
        int sum = 0;
        for (int k=i; k<=j; k++)
            sum += a[k];
        if (sum > maxSum)
            maxSum = sum;
    }
}
```

11

Update maxSum

Time complexity: $O(N^3)$

$$\begin{aligned} \sum_{i=0}^{n-1} \sum_{j=i}^{n-1} (j-i+1) &= \sum_{i=0}^{n-1} (1+2+\dots+(n-i)) = \sum_{i=0}^{n-1} \frac{(n-i)(n-i+1)}{2} \\ &= \frac{1}{2} \sum_{k=1}^n k(k+1) = \frac{1}{2} \left[\sum_{k=1}^n k^2 + \sum_{k=1}^n k \right] = \frac{1}{2} \left[\frac{n(n+1)(2n+1)}{6} + \frac{n(n+1)}{2} \right] \\ &= \frac{n^3}{6} + \frac{n^2}{2} + \frac{n}{3} \end{aligned}$$

That may reach time limit exceed in leetcode (TLE)

Space complexity: $O(1)$

2. Specific:

For example:
We have an array

3	-1	2	5
---	----	---	---

$\text{maxSum} = -\infty$
 $i = 0$

$j = i = 0$
 $\text{sum} = 0$

$k = 0$

initial values

Loop 1:

$j = 0$

$k = 0$

$\text{arr}[k] = \text{arr}[0] = 3$
 $\text{sum} = \text{sum} + \text{arr}[k]$
 $= 3$
 $\text{maxSum} = 3$

$j = j + 1 = 1$

$k = 0$

$\text{arr}[k] = \text{arr}[0] = 3$
 $\text{sum} = \text{sum} + \text{arr}[k]$
 $= 3$
 $\text{maxSum} = 3$

o o o

$j = 3$

$k = 0$

$k = 1$

$\text{arr}[k] = -1$
 $\text{sum} = 2$
 $\text{maxSum} = 3$

$k = 1$

```
int maxSum = -∞;
for (int i=0; i<n; i++) {
    for (int j=i; j<n; j++) {
        int sum = 0;
        for (int k=i; k<=j; k++)
            sum += a[k];
        if (sum > maxSum)
            maxSum = sum;
    }
}
```

11

$$\begin{aligned} \text{arr}[k] &= \text{arr}[0] = 3 \\ \text{sum} &= \text{sum} + \text{arr}[k] \\ &= 3 \\ \text{maxSum} &= 3 \end{aligned}$$

$$k = 2$$

$$\begin{aligned} \text{arr}[k] &= 2 \\ \text{sum} &= 4 \\ \text{maxSum} &= 4 \end{aligned}$$

$$\begin{aligned} \text{arr}[k] &= -1 \\ \text{sum} &= 2 \\ \text{maxSum} &= 3 \end{aligned}$$

$$k = 3$$

$$\begin{aligned} \text{arr}[k] &= 5 \\ \text{sum} &= 9 \\ \text{maxSum} &= 9 \end{aligned}$$

Continue with $i = 1 \dots (n-1)$

In this example we can solve it just one i loop :D but I think that may an easy way to make sense about that problem. And also we have too many redundant calculations. Can we brainstorm a more optimized approach to solve this problem?

II. USING BRUTE FORCE - BETTER APPROACH:

$$\text{maxSum} = -\infty$$

$$i = 0:$$

$$\text{sum} = 0$$

$$\begin{aligned} j &= 0 \\ \text{sum} &= 0 + a[0] \\ &= 3 \\ \text{maxSum} &= 3 \end{aligned}$$

$$\begin{aligned} j &= 1 \\ \text{sum} &= 3 - 1 \\ &= 2 \\ \text{maxSum} &= 3 \end{aligned}$$

$$\begin{aligned} j &= 2 \\ \text{sum} &= 2 + 2 \\ &= 4 \\ \text{maxSum} &= 4 \end{aligned}$$

$$\begin{aligned} j &= 3 \\ \text{sum} &= 4 + 5 \\ &= 9 \\ \text{maxSum} &= 9 \end{aligned}$$

```
int maxSum = INT_MIN; // Option: maxSum = a[0]
for (int i=0; i<n; i++) {
    int sum = 0;
    for (int j=i; j<n; j++) {
        sum += a[j];
        if (sum > maxSum)
            maxSum = sum;
    }
}
```

14

Why is this better?

Because k -loop was removed. And now, we can reuse the recently sum for the next calculations.

$$\sum_{k=i}^j a[k] = a[j] + \sum_{k=i}^{j-1} a[k]$$

III. USING DYNAMIC PROGRAMMING - KANADE'S ALGORITHM: